

Centre National de Formation en Statistique,
Démographie et Planification
Brazzaville, République du Congo

Introduction au Machine Learning

Chapitre 2 – Apprentissage supervisé : régression

Moindres carrés, régularisation, forêts et boosting

Rois Céti DIMBAMBA
Ingénieur statisticien économiste · Actuaire · CV en ligne

Licence 3 · Semestre 5 · UE Informatique IV
Année académique 2026-2027 · 16 octobre 2026

Plan et objectifs

1. Le problème de régression
2. Les moindres carrés
3. Plusieurs variables
4. Quand les moindres carrés déraillent
5. Régulariser : Ridge et Lasso
6. Arbres, forêts et boosting
7. En Python
8. TP en salle

Objectifs de la séance

- Écrire un modèle linéaire et l'ajuster par moindres carrés, à la main et en Python.
- Lire les coefficients et le R^2.
- Reconnaître le sur-apprentissage quand le modèle a trop de variables.
- Régulariser avec Ridge et Lasso, et choisir la pénalité.
- Utiliser forêts aléatoires et gradient boosting en régression.

Prérequis : chapitre 0 (droite des moindres carrés, gradient, matrices) et chapitre 1 (risque, validation croisée).

1

Le problème de régression

Prédire une quantité

Définition 2.1 (Problème de régression). Un problème supervisé est un problème de *régression* quand la cible y est un nombre réel. On mesure l'erreur par la perte quadratique : le risque d'un prédicteur f est $R(f) = \mathbb{E}\big[(Y - f(X))^2\big]$.

Exemples : revenu d'un ménage, nombre d'enfants d'une femme, rendement d'une parcelle, durée d'un chômage.

Fil rouge (chiffres fictifs) : 5 femmes.

années d'études x	0	3	6	9	12
nombre d'enfants y	6	5	5	3	1

Rappel du chapitre 1. Le meilleur prédicteur possible est la moyenne conditionnelle $f^*(x) = \mathbb{E}[Y \mid X = x]$. Elle est inconnue : on cherche à l'approcher à partir des données.

Le modèle linéaire

Définition 2.2 (Prédicteur linéaire). Avec d variables explicatives $x = (x_1, \ldots, x_d)$, un *prédicteur linéaire* est de la forme

$$f_\theta(x) = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d,$$

où $\theta = (\theta_0, \theta_1, \ldots, \theta_d)$ sont les *coefficients* : θ_0 est la constante, θ_j l'effet de la variable x_j.

- **Simple** : $d + 1$ nombres à estimer, que l'on peut interpréter.
- **Souvent une bonne approximation** de $\mathbb{E}[Y \mid X = x]$, au moins localement.
- **Plus souple qu'il n'y paraît** : on peut y mettre x^2, $\log(\text{revenu})$, des indicatrices de catégories… « Linéaire » veut dire linéaire *en* θ.

2

Les moindres carrés

Moindres carrés avec une variable

Définition 2.3 (Estimateur des moindres carrés ordinaires (MCO)). Les coefficients $(\hat{a}, \hat{b})$ des MCO sont ceux qui minimisent le risque empirique quadratique

$$\widehat{R}_n(a,b) = \frac{1}{n}\sum_{i=1}^{n} \big(y_i - a - b x_i\big)^2.$$

Proposition 2.4 (droite des moindres carrés, chapitre 0)

Avec $s_{xy} = \frac{1}{n}\sum_i (x_i - \bar{x})(y_i - \bar{y})$ et $s_x^2 = \frac{1}{n}\sum_i (x_i - \bar{x})^2 > 0$,

$$\hat{b} = s_{xy}/s_x^2, \qquad \hat{a} = \bar{y} - \hat{b}\,\bar{x}.$$

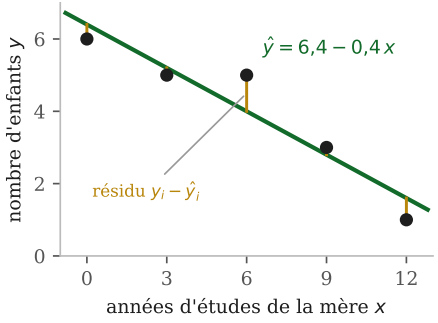
Preuve (rappel). $\partial \widehat{R}_n/\partial a = 0$ donne $a = \bar{y} - b\bar{x}$; il reste $s_y^2 - 2b\,s_{xy} + b^2 s_x^2$, minimal en $b = s_{xy}/s_x^2$. □

Le calcul sur le fil rouge

$\bar{x} = 6$, $\bar{y} = 4$.

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	produit	carré
0	6	-6	2	-12	36
3	5	-3	1	-3	9
6	5	0	1	0	0
9	3	3	-1	-3	9
12	1	6	-3	-18	36
			moyenne	$s_{xy} = -7{,}2$	$s_x^2 = 18$

$\hat{b} = \frac{-7{,}2}{18} = -0{,}4$ et $\hat{a} = 4 - (-0{,}4) \times 6 = 6{,}4$.



Lecture. Une année d'études de plus est associée, en moyenne, à $0{,}4$ enfant de moins. C'est une *association* observée, pas un effet causal démontré.

Résidus et coefficient de détermination

Définition 2.5 (Résidus, R^2). Les *résidus* sont $e_i = y_i - \hat{y}_i$, où $\hat{y}_i = \hat{a} + \hat{b}x_i$. Le *coefficient de détermination* est

$$R^2 = 1 - \frac{\sum_i e_i^2}{\sum_i (y_i - \bar{y})^2} \in [0, 1].$$

- Les résidus des MCO sont de somme nulle : c'est l'équation $\partial \widehat{R}_n / \partial a = 0$.
- Fil rouge : $\hat{y} = (6{,}4;\ 5{,}2;\ 4;\ 2{,}8;\ 1{,}6)$, résidus $(-0{,}4;\ -0{,}2;\ 1;\ 0{,}2;\ -0{,}6)$, de somme nulle.
- $\sum_i e_i^2 = 1{,}6$ et $\sum_i (y_i - \bar{y})^2 = 16$, d'où $R^2 = 1 - \frac{1{,}6}{16} = 0{,}9$.

Lecture et piège. La droite « explique » $90\,\%$ de la dispersion du nombre d'enfants. Mais ce R^2 est calculé sur les données d'apprentissage : il augmente toujours quand on ajoute des variables. On juge un modèle sur un échantillon test (chapitre 1).

3

Plusieurs variables

L'écriture matricielle

On range les données dans une matrice X à n lignes et $d+1$ colonnes, la première colonne étant remplie de 1 (pour la constante), et les cibles dans un vecteur $y \in \mathbb{R}^n$. Les prédictions s'écrivent $\hat{y} = X\theta$ et le risque empirique

$$\widehat{R}_n(\theta) = \frac{1}{n}\|y - X\theta\|^2 = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \theta_0 - \theta_1 x_{i1} - \cdots - \theta_d x_{id}\right)^2.$$

Théorème 2.6 (Moindres carrés : équations normales)

Si la matrice $X^\top X$ est inversible, le risque empirique atteint son minimum en un unique point,

$$\hat{\theta} = (X^\top X)^{-1} X^\top y.$$

Preuve (idée). Au minimum, le gradient est nul : $\nabla \widehat{R}_n(\theta) = -\frac{2}{n} X^\top (y - X\theta) = 0$, soit $X^\top X\,\theta = X^\top y$ (calcul du gradient admis). □

Vérification sur le fil rouge

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 3 \\ 1 & 6 \\ 1 & 9 \\ 1 & 12 \end{pmatrix}, \quad X^\top X = \begin{pmatrix} 5 & 30 \\ 30 & 270 \end{pmatrix}, \quad X^\top y = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \end{pmatrix} = \begin{pmatrix} 20 \\ 84 \end{pmatrix}.$$

Le déterminant vaut $5 \times 270 - 30 \times 30 = 450$, donc (chapitre 0)

$$(X^\top X)^{-1} = \frac{1}{450} \begin{pmatrix} 270 & -30 \\ -30 & 5 \end{pmatrix}, \qquad \hat{\theta} = \frac{1}{450} \begin{pmatrix} 270 \times 20 - 30 \times 84 \\ -30 \times 20 + 5 \times 84 \end{pmatrix} = \begin{pmatrix} 6{,}4 \\ -0{,}4 \end{pmatrix}.$$

À retenir

Avec une seule variable, la formule matricielle redonne la droite des moindres carrés. Avec d variables, c'est le même calcul : l'ordinateur résout le système $X^\top X\theta = X^\top y$.

Lire les coefficients

Définition 2.7 (Interprétation d'un coefficient). $\hat{\theta}_j$ est la variation moyenne de la prédiction quand x_j augmente d'une unité, *les autres variables restant fixées*.

Variable qualitative. On la code par des *indicatrices*, en laissant une catégorie de référence :

$$x_{\text{rural}} = \begin{cases} 1 & \text{si rural} \\ 0 & \text{si urbain.} \end{cases}$$

Dans $\hat{y} = \hat{\theta}_0 + \hat{\theta}_1 \,\text{études} + \hat{\theta}_2\, x_{\text{rural}}$, le coefficient $\hat{\theta}_2$ est l'écart moyen entre rurales et urbaines *à niveau d'études égal*.

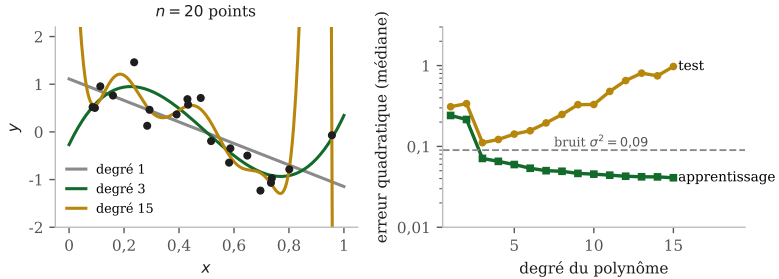
Précautions.

- Un coefficient dépend de l'unité de x_j (années, mois, FCFA).
- Il dépend aussi des autres variables présentes dans le modèle.
- Association n'est pas causalité.

4

Quand les moindres carrés déraillent

Trop de variables : le sur-apprentissage revient



$Y = \sin(2\pi X) + \varepsilon$, $\sigma = 0{,}3$; polynômes de degré 1 à 15 (variables $x, x^2, \ldots, x^d$) ajustés par MCO. Le degré 1 est trop rigide, le degré 3 est le meilleur sur le test ; le degré 15 colle aux points et oscille entre eux : l'erreur test explose.

Autres symptômes : si $d+1 > n$, $X^\top X$ n'est pas inversible (une infinité de solutions annulent l'erreur d'apprentissage) ; si deux variables sont très corrélées, leurs coefficients deviennent instables.

5

Régulariser : Ridge et Lasso

La régression Ridge

Définition 2.8 (Ridge). Pour une pénalité $\lambda \geq 0$, les coefficients Ridge minimisent

$$\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \theta_0 - \theta_1 x_{i1} - \cdots - \theta_d x_{id}\right)^2 + \lambda \sum_{j=1}^{d} \theta_j^2.$$

- Le premier terme veut coller aux données, le second veut des coefficients petits. On ne pénalise pas la constante θ_0.
- $\lambda = 0$: on retrouve les MCO. Quand λ grandit, les $\hat{\theta}_j$ se rapprochent de 0.
- La solution existe et est unique pour tout $\lambda > 0$, même si $d + 1 > n$ (admis).

À retenir

Régulariser, c'est accepter un peu de biais pour réduire beaucoup la variance (chapitre 1).

Ridge avec une variable : un calcul complet

Proposition 2.9

Avec une seule variable, la pente Ridge vaut

$$\hat{b}_\lambda = \frac{s_{xy}}{s_x^2 + \lambda}, \qquad \text{et} \qquad \hat{a}_\lambda = \bar{y} - \hat{b}_\lambda \, \bar{x}.$$

Preuve. La pénalité ne dépend pas de a : comme pour les MCO, $a = \bar{y} - b\bar{x}$. Il reste à minimiser $s_y^2 - 2b\, s_{xy} + b^2 s_x^2 + \lambda b^2$, dont la dérivée $-2s_{xy} + 2b\,(s_x^2 + \lambda)$ s'annule en $b = s_{xy}/(s_x^2 + \lambda)$.□

Exemple. [Fil rouge] $s_{xy} = -7{,}2$, $s_x^2 = 18$. Pour $\lambda = 4$: $\hat{b}_4 = \frac{-7{,}2}{22} \approx -0{,}327$ au lieu de $-0{,}4$. Le dénominateur grossit : la pente est *rétrécie* vers 0, sans jamais l'atteindre.

La régression Lasso

Définition 2.10 (Lasso). Les coefficients Lasso minimisent le même critère que Ridge, avec $\lambda \sum_{j=1}^{d} |\theta_j|$ à la place de $\lambda \sum_{j=1}^{d} \theta_j^2$: on pénalise les valeurs absolues au lieu des carrés.

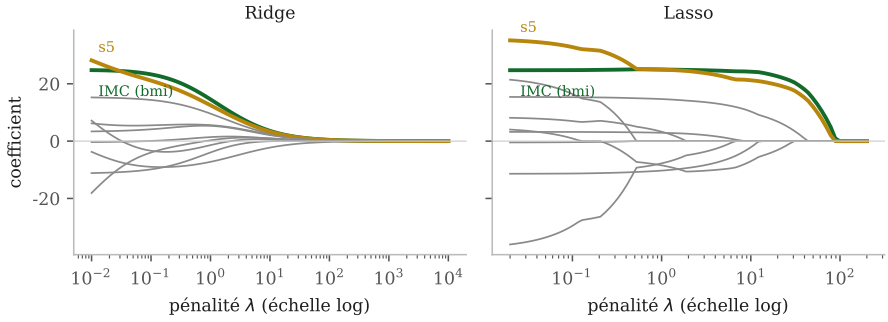
Proposition 2.11 (admise)

Avec une seule variable, $\hat{b}_\lambda = \text{signe}(s_{xy}) \times \dfrac{\max\left(|s_{xy}| - \lambda/2,\ 0\right)}{s_x^2}$: la pente est *exactement nulle* dès que $\lambda \geq 2|s_{xy}|$.

λ	0	4	10	$14{,}4$	20
pente Ridge	$-0{,}400$	$-0{,}327$	$-0{,}257$	$-0{,}222$	$-0{,}189$
pente Lasso	$-0{,}400$	$-0{,}289$	$-0{,}122$	$\mathbf{0}$	$\mathbf{0}$

Ridge rétrécit tous les coefficients ; Lasso en annule certains : il *sélectionne* les variables.

Les chemins de régularisation



Données `diabetes` de scikit-learn : 442 patients, 10 variables standardisées (âge, sexe, IMC, tension, 6 mesures sanguines `s1` à `s6`) ; y = progression de la maladie un an plus tard. Chaque courbe suit un coefficient quand λ augmente. Avec le Lasso, les coefficients s'annulent un à un : vers $\lambda = 40$, il ne reste que 3 variables : l'IMC, la tension (`bp`) et `s5`.

Bien utiliser Ridge et Lasso

1. **Standardiser les variables** (moyenne 0, écart type 1) : la pénalité compare des coefficients, qui dépendent des unités. Un revenu en FCFA ou en millions de FCFA ne serait pas pénalisé de la même façon.
2. **Choisir λ par validation croisée** (chapitre 1) : pour chaque λ d'une grille, on calcule l'erreur de validation croisée ; on garde le λ qui la minimise, puis on réapprend sur tout l'échantillon d'apprentissage.
3. **Choisir la pénalité** : Ridge si beaucoup de variables utiles et corrélées ; Lasso si l'on pense que peu de variables comptent, ou pour un modèle plus lisible.

Conventions de scikit-learn. Le paramètre s'appelle `alpha`, avec d'autres échelles : `Ridge(alpha=`$n\lambda$`)` et `Lasso(alpha=`$\lambda/2$`)`. `RidgeCV` et `LassoCV` font la validation croisée.

6

Arbres, forêts et boosting

L'arbre de régression

Définition 2.12 (Arbre de régression). Un arbre découpe l'espace des x en zones par des questions du type « $x_j \leq s$? ». Dans chaque zone (*feuille*), il prédit la *moyenne* des y_i de la zone : la meilleure constante (chapitre 1).

Choisir la coupure. On essaie chaque seuil et l'on garde celui qui minimise la somme des carrés des écarts à la moyenne, dans les deux zones.

Fil rouge, une seule coupure : on retient $x \leq 7{,}5$. L'arbre prédit $\frac{6+5+5}{3} \approx 5{,}33$ enfants à gauche et $\frac{3+1}{2} = 2$ à droite.

seuil	zones	somme des carrés
1,5	$\{6\} \mid \{5,5,3,1\}$	11
4,5	$\{6,5\} \mid \{5,3,1\}$	8,5
7,5	$\{6,5,5\} \mid \{3,1\}$	**2,67**
10,5	$\{6,5,5,3\} \mid \{1\}$	4,75

On recommence dans chaque zone. Un arbre profond sur-apprend (chapitre 1) : il est très *instable*.

Forêt aléatoire : la moyenne de nombreux arbres

Définition 2.13 (Bagging, forêt aléatoire).

1. On tire B échantillons *bootstrap* : n individus tirés *avec remise* dans l'échantillon.
2. Sur chacun, on apprend un arbre profond $\hat{f}_b$.
3. On prédit par la moyenne $\hat{f}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}_b(x)$.

Une *forêt aléatoire* ajoute du hasard : à chaque coupure, l'arbre ne regarde qu'une partie des variables, tirée au sort.

- Chaque arbre est instable, mais leurs erreurs se compensent : la moyenne varie beaucoup moins (calcul au chapitre 3).
- Réglages : nombre d'arbres B (plusieurs centaines), taille minimale des feuilles, nombre de variables tirées.

Gradient boosting : corriger pas à pas

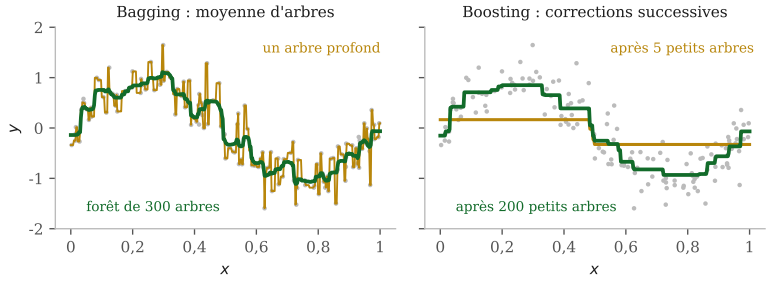
Définition 2.14 (Gradient boosting en régression). On part de $F_0(x) = \bar{y}$. Pour $m = 1, \dots, M$:

1. on calcule les résidus $r_i = y_i - F_{m-1}(x_i)$: ce qui reste à expliquer ;
2. on ajuste un *petit* arbre h_m sur les couples (x_i, r_i) ;
3. on corrige un peu : $F_m(x) = F_{m-1}(x) + \nu\, h_m(x)$, avec un taux d'apprentissage $0 < \nu \leq 1$.

Fil rouge, arbres à une coupure, $\nu = 0{,}5$: $F_0 = 4$; résidus $(2; 1; 1; -1; -3)$. Le meilleur arbre coupe en $x \leq 7{,}5$ et prédit $\frac{4}{3}$ à gauche, -2 à droite.

$$F_1 = (4{,}67;\ 4{,}67;\ 4{,}67;\ 3;\ 3), \qquad F_2 = (4{,}92;\ 4{,}92;\ 4{,}92;\ 3{,}25;\ 2) \quad \text{pour } y = (6; 5; 5; 3; 1).$$

Bagging et boosting en image



	Forêt (bagging)	**Gradient boosting**
Arbres	profonds, appris en parallèle	petits, appris l'un après l'autre
Ce qui baisse	la variance	le biais
Réglages clés	B, taille des feuilles	ν, nombre d'arbres M, profondeur

Quel modèle choisir ?

Modèle	Points forts	Limites
MCO	simple, rapide, coefficients lisibles	sur-apprend avec beaucoup de variables
Ridge, Lasso	beaucoup de variables, variables corrélées ; Lasso sélectionne	relation supposée linéaire ; standardiser
Forêt, boosting	relations non linéaires, interactions, peu de préparation	moins lisibles ; demandent plus de données

À retenir

Aucun modèle ne gagne toujours. Sur une relation en sinus, la forêt et le boosting font nettement mieux que la droite (erreur $0{,}32$ contre $0{,}54$) ; sur les données `diabetes`, la régression linéaire bat la forêt. On compare toujours sur un échantillon test ou par validation croisée.

7

En Python

Moindres carrés : à la main puis avec scikit-learn

```python
import numpy as np
from sklearn.linear_model import LinearRegression

# Fil rouge : années d'études de la mère (x), nombre d'enfants (y)
x = np.array([0, 3, 6, 9, 12]); y = np.array([6, 5, 5, 3, 1])
X = np.column_stack([np.ones(5), x])            # colonne de 1 pour la constante
theta = np.linalg.solve(X.T @ X, X.T @ y)       # équations normales
print(theta)                                    # [ 6.4 -0.4]

f = LinearRegression().fit(x.reshape(-1, 1), y) # même calcul avec scikit-learn
print(f.intercept_, f.coef_)                    # 6.4 [-0.4]
print(round(f.score(x.reshape(-1, 1), y), 3))   # R^2 = 0.9
```

`np.linalg.solve` résout le système $X^\top X\theta = X^\top y$ sans calculer l'inverse, ce qui est plus précis.

Ridge et Lasso avec validation croisée

```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.metrics import root_mean_squared_error

X, y = load_diabetes(return_X_y=True, as_frame=True)   # 442 patients, 10 variables
Xa, Xt, ya, yt = train_test_split(X, y, test_size=0.3, random_state=0)
modeles = {"MCO": LinearRegression(),
           "Ridge": RidgeCV(alphas=np.logspace(-3, 3, 50)),
           "Lasso": LassoCV(cv=5, random_state=0)}
for nom, m in modeles.items():
    f = make_pipeline(StandardScaler(), m).fit(Xa, ya)  # standardiser puis ajuster
    print(nom, round(root_mean_squared_error(yt, f.predict(Xt)), 1))
coef = f[-1].coef_                                      # coefficients du Lasso
print(list(X.columns[coef == 0]))                       # variables écartées
# MCO 55.7 | Ridge 55.4 | Lasso 55.8 | écartées : ['s2', 's4', 's6']
```

Les trois se valent (environ 55 points d'erreur) ; le Lasso n'utilise que 7 variables sur 10.

Forêt et boosting contre la régression linéaire

```python
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
rng = np.random.default_rng(0)

def simuler(n):                                    # relation non linéaire
    x = rng.uniform(0, 1, (n, 1))
    return x, np.sin(2 * np.pi * x[:, 0]) + 0.3 * rng.standard_normal(n)

X, y = simuler(300); Xt, yt = simuler(5_000)
modeles = {
    "linéaire": LinearRegression(),
    "forêt": RandomForestRegressor(n_estimators=200, min_samples_leaf=5,
                                   random_state=0),
    "boosting": GradientBoostingRegressor(n_estimators=200, learning_rate=0.05,
                                          max_depth=2, random_state=0)}
for nom, f in modeles.items():
    print(nom, round(root_mean_squared_error(yt, f.fit(X, y).predict(Xt)), 3))
# linéaire 0.541 | forêt 0.316 | boosting 0.317
```

Le bruit a pour écart type $\sigma = 0{,}3$: la forêt et le boosting en sont tout près, la droite non.

8

TP en salle

TP2 – Moindres carrés, régularisation, ensembles

TP2 – à faire en salle — 30 min · `notebooks/TP2.ipynb` (Colab)

1. **Moindres carrés.** Simuler $n = 100$ points de $Y = 2 + 3X + \varepsilon$; calculer $\hat{a}$, $\hat{b}$ par les formules, puis par les équations normales ; comparer avec `LinearRegression` et calculer le R^2.
2. **Chemin du Lasso** (données `diabetes`) : quelles variables restent quand λ grandit ?
3. **Choix de λ** par validation croisée ; comparer les erreurs test des MCO, de Ridge et du Lasso.
4. **Boosting à la main** : programmer deux étapes sur le fil rouge et retrouver F_1 et F_2 du cours.
5. **Ensembles** : comparer régression linéaire, forêt et gradient boosting sur la relation en sinus, puis sur `diabetes`.

Résumé

- Régression : prédire un nombre ; on approche $\mathbb{E}[Y \mid X = x]$ en minimisant l'erreur quadratique.
- Moindres carrés : $\hat{b} = s_{xy}/s_x^2$ avec une variable, $\hat{\theta} = (X^\top X)^{-1} X^\top y$ en général ; le R^2 mesure l'ajustement, pas la qualité de prédiction.
- Avec trop de variables, les MCO sur-apprennent et deviennent instables.
- Ridge rétrécit les coefficients, Lasso en annule certains ; on standardise et l'on choisit λ par validation croisée.
- Arbres : moyenne par zone. Forêt : moyenne d'arbres (moins de variance). Boosting : petits arbres qui corrigent les résidus (moins de biais).
- Aucun modèle n'est toujours le meilleur : on compare sur des données test.